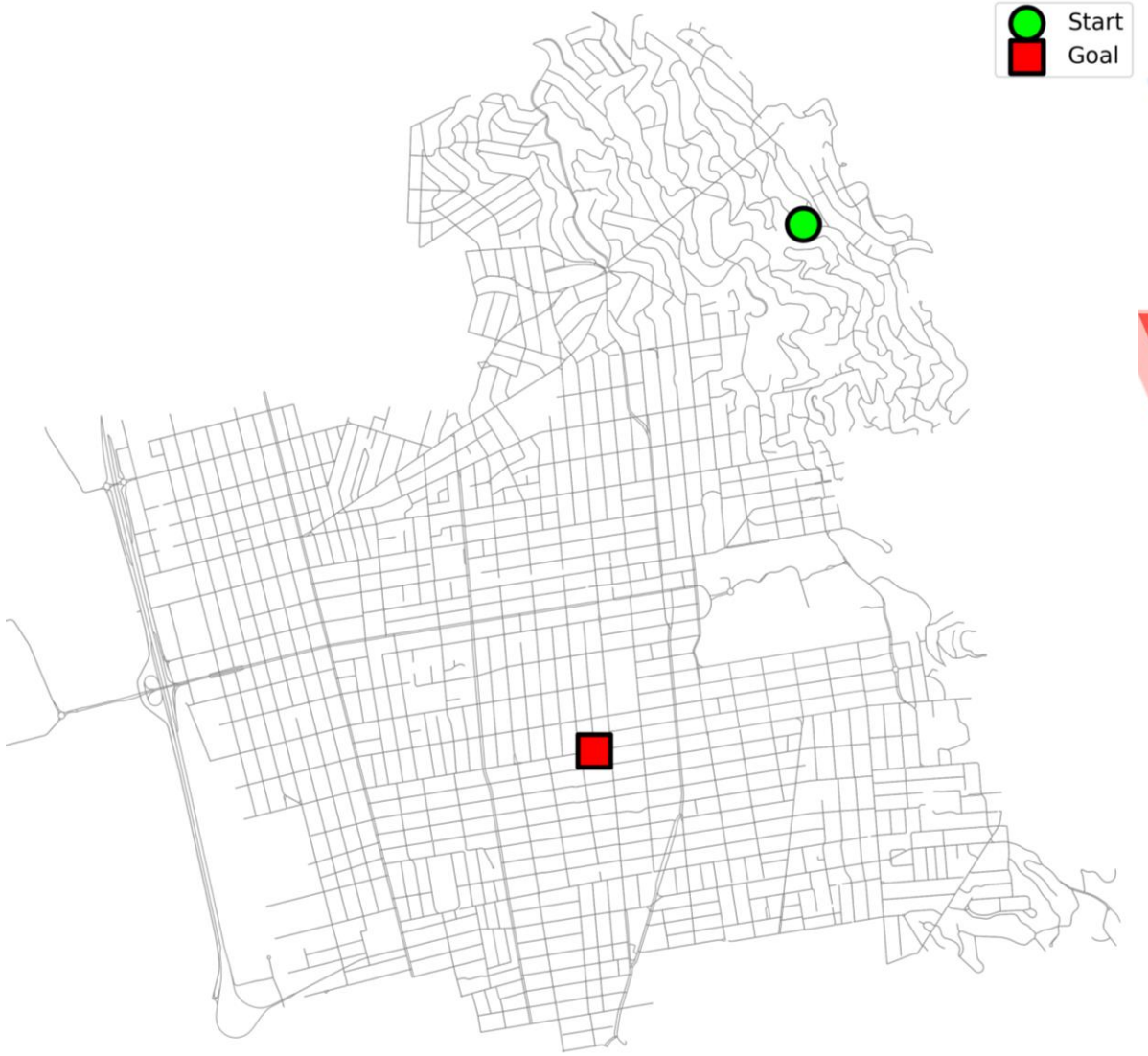
香港城市大學
City University of Hong Kong
Innovating into the Future

CS5493: Tutorial 11
Route Planning

Route Planning Mission

Given a vector map of roads of a city (Berkeley, California in this scenario), a start point on the map and a destination, how to decide what route to take to achieve highest efficiency?

Now let's consider a simple scenario: shortest path.



Route Planning Mission

For given information, we have multiple methods to get the shortest path, but in this tutorial, we will focus on two algorithms and their implementation:

1. Dijkstra's Algorithm
2. A* (A-Star) Algorithm

Dijkstra’s Algorithm

Core Idea:
Maintain a priority queue, always selecting the closest unvisited node from the origin for expansion until reaching the goal node.

Light red areas: Explored nodes (not in final path)
Dark red path: Optimal path found

Notice uniform expansion pattern in all directions

Dijkstra's Algorithm

Key Operations:

- Pop minimum cost node
- Update neighbor costs
- Record predecessor relations
- Mark nodes as visited

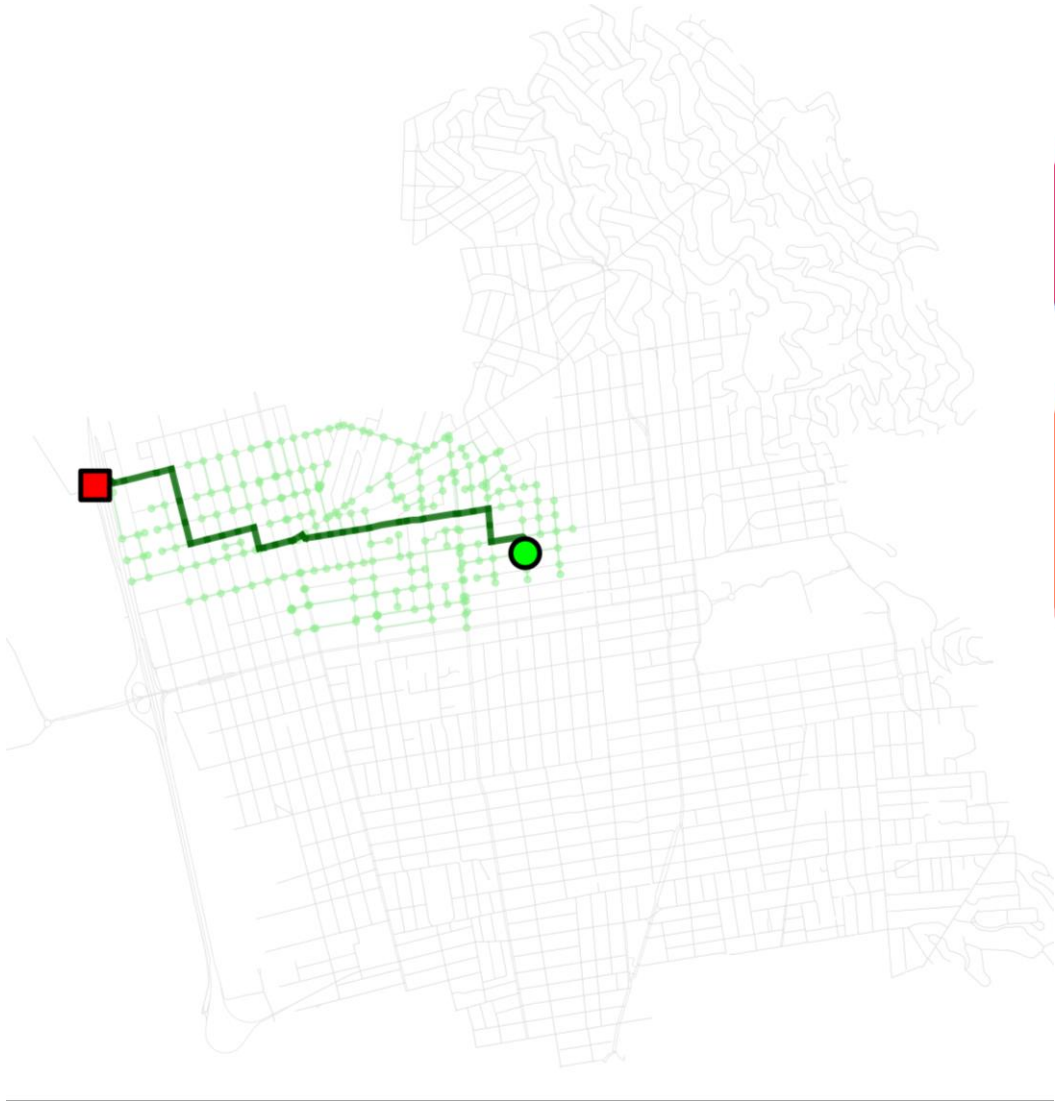
Algorithm 1 DIJKSTRA(G, s, t)

```
open ← MinHeap()
closed ← Set()
predecessors ← Dict()
open.push(s, 0)
while !open.isEmpty() do
    u, uCost ← open.pop()
    if isGoal(u) then
        return extractPath(u, predecessors)
    for all v ∈ u.successors do
        if v ∈ closed then
            continue
        uvCost ← edgeCost(G, u, v)
        if v ∈ open then
            if uCost + uvCost < open[v] then
                open[v] ← uCost + uvCost
                predecessors[v] ← u
        else
            open.push(v, uCost + uvCost)
            predecessors[v] ← u
    closed.add(u)
```

A* Algorithm

Why more algorithms?

In Dijkstra, we notice that search is towards all directions, but actual shortest path is more likely existing in the direction towards destination! So a heuristic function can be used to represent this.



A* Algorithm

Heuristic Function

Heuristic estimation can be used to estimate the distance from current spot to the destination:

In Dijkstra:

$$f(n) = g(n)$$

In A*:

$$f(n) = g(n) + \boldsymbol{h(n)}$$

*g(n):*Actual cost from origin to node n

*h(n):*Heuristic estimate from node n to goal

*f(n):*Total estimated cost

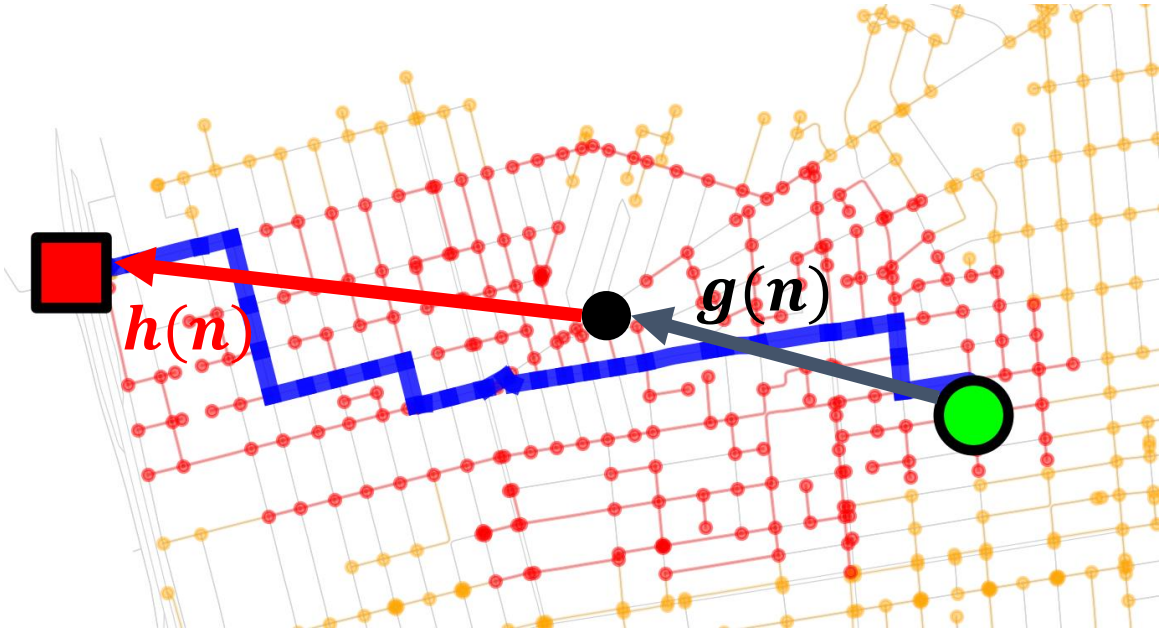
A* Algorithm

Heuristic Function

In the tutorial we use Euclidean Distance between current node and destination as heuristic function:

$$h(n) = \sqrt{(d_x - n_x)^2 + (d_y - n_y)^2 + (d_z - n_z)^2}$$

d is the destination.



A* Algorithm

Heuristic Properties of Euclidean Distance
Admissible: $h(n) \leq h*(n)$, never overestimates actual distance
Consistent: Triangle inequality holds
These two guarantees finding optimal solution

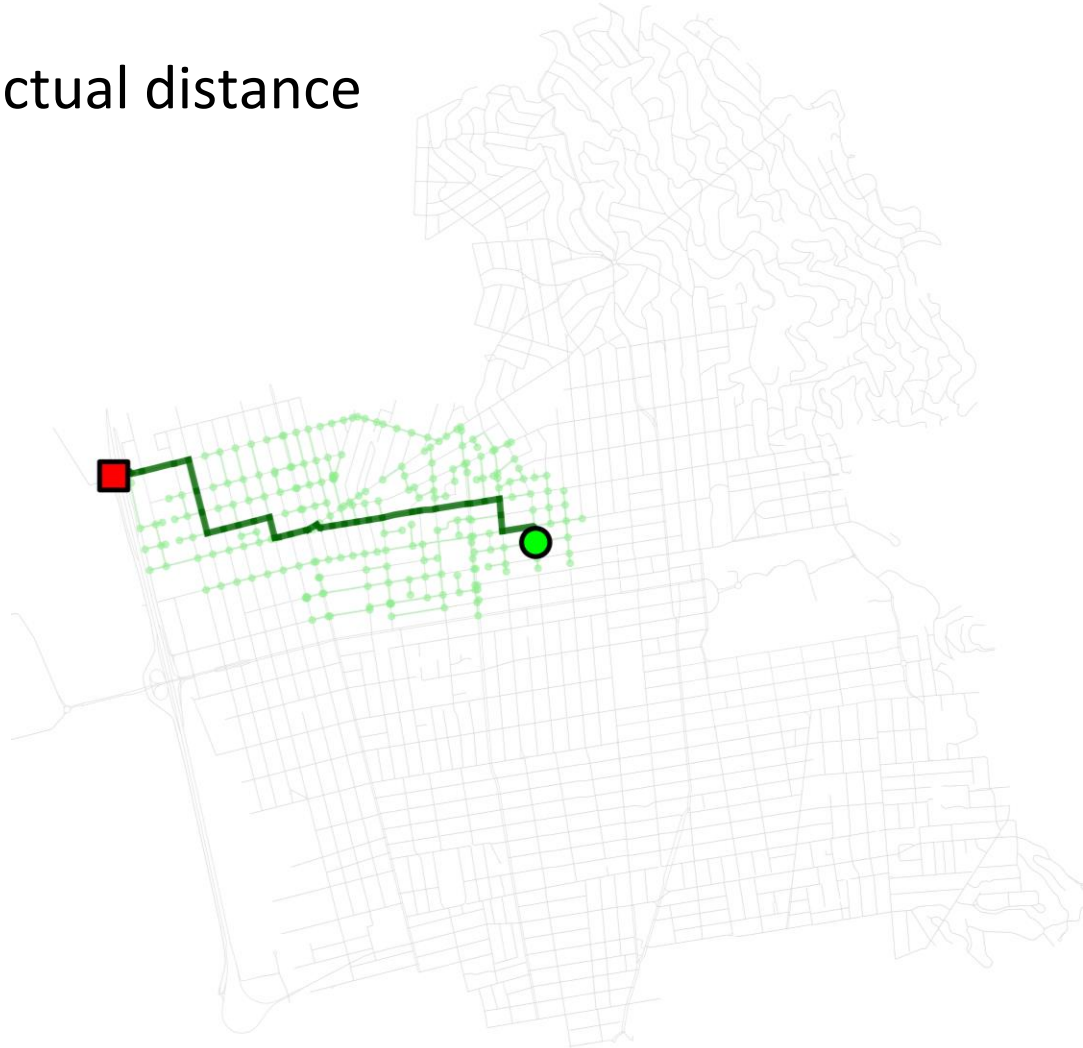
Light green areas:

Explored nodes (not in final path)

Dark green path:

Optimal path found

Notice directed expansion towards goal



A* Algorithm

Algorithm 1 A*(G, s, t)

```
open ← MinHeap()
closed ← Set()
predecessors ← Dict()
costs ← Dict()
open.push(s, 0)
costs[s] ← 0
while !open.isEmpty() do
    u ← open.pop()
    uCost ← costs[u]
    if isGoal(u) then
        return extractPath(u, predecessors)
    for all v ∈ u.successors do
        if v ∈ closed then
            continue
        uvCost ← edgeCost(G, u, v)
        if v ∈ open then
            if uCost + uvCost + h(v) < open[v] then
                open[v] ← uCost + uvCost + h(v)
                costs[v] ← uCost + uvCost
                predecessors[v] ← u
        else
            open.push(v, uCost + uvCost + h(v))
            costs[v] ← uCost + uvCost
            predecessors[v] ← u
    closed.add(u)
```

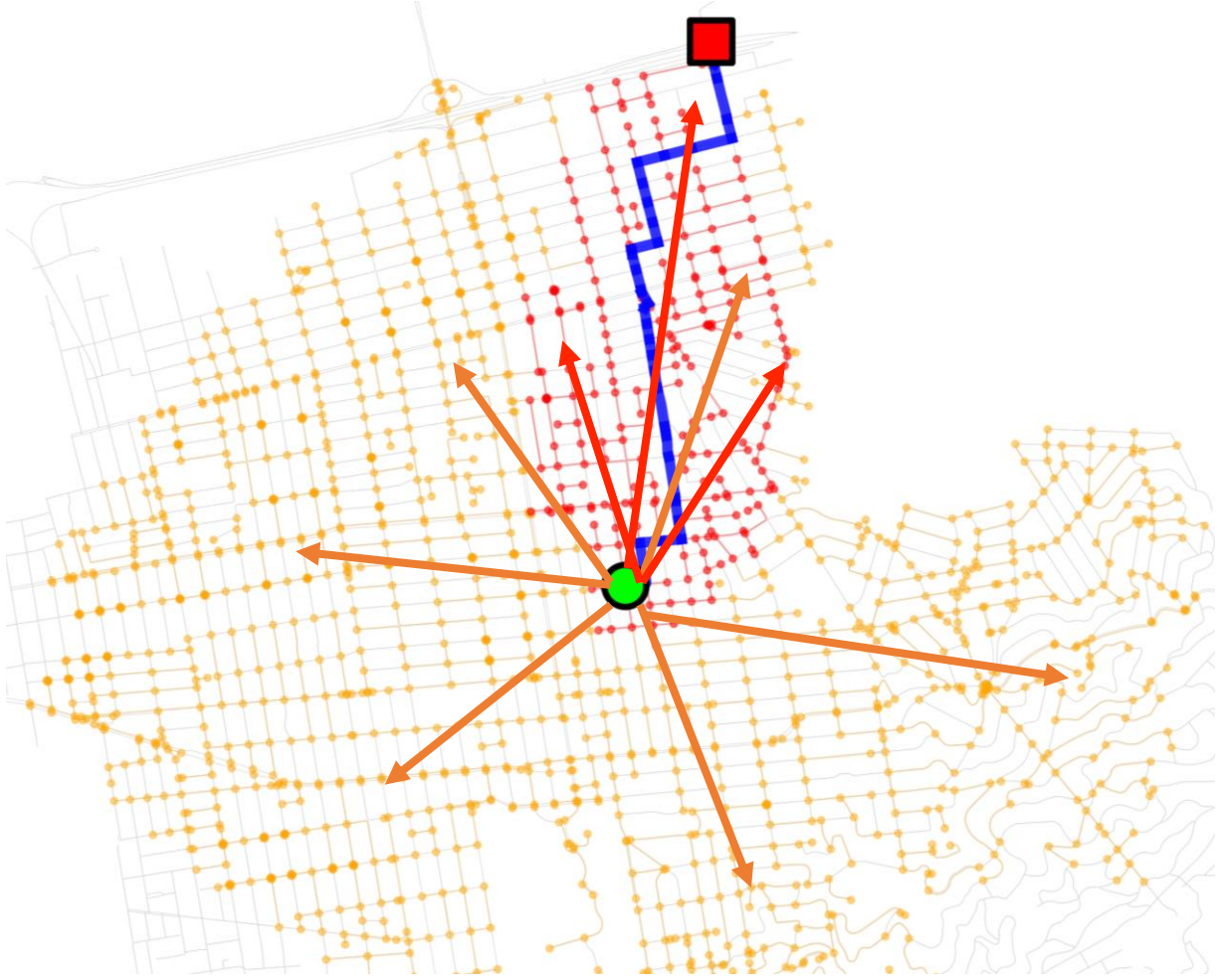
Algorithm Comparison

In Dijkstra:

$$f(n) = g(n)$$

In A*:

$$f(n) = g(n) + \boldsymbol{h(n)}$$



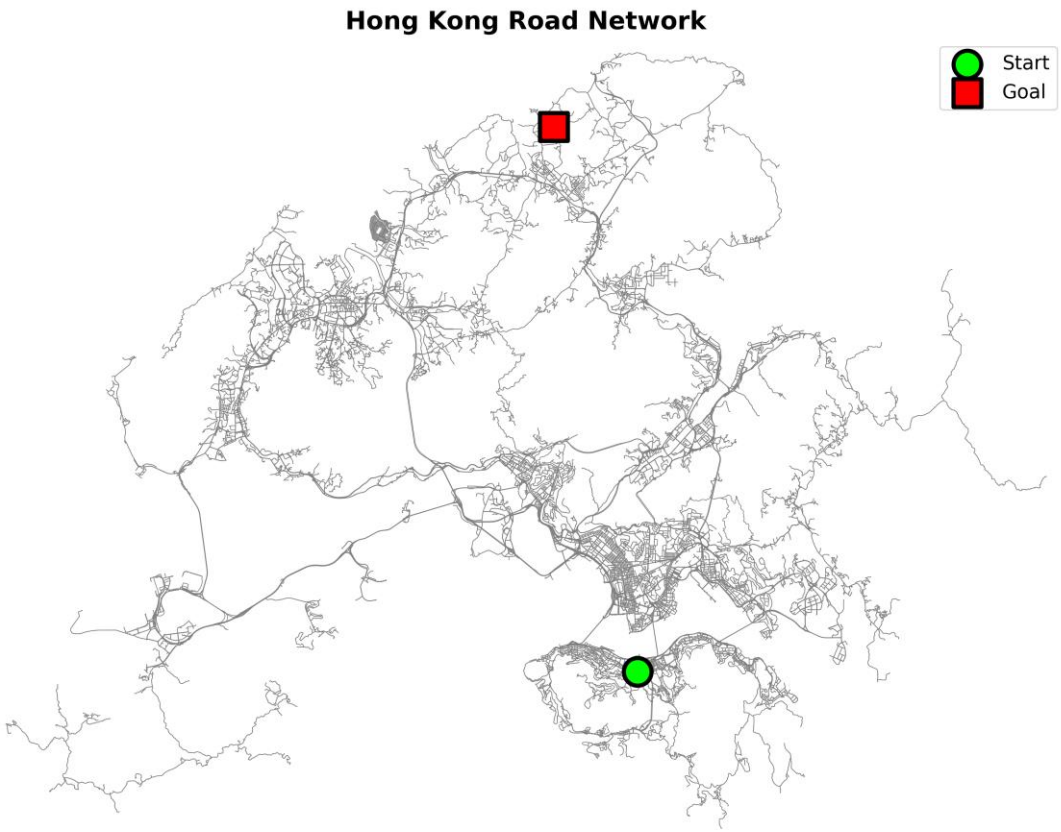
Algorithm Comparison

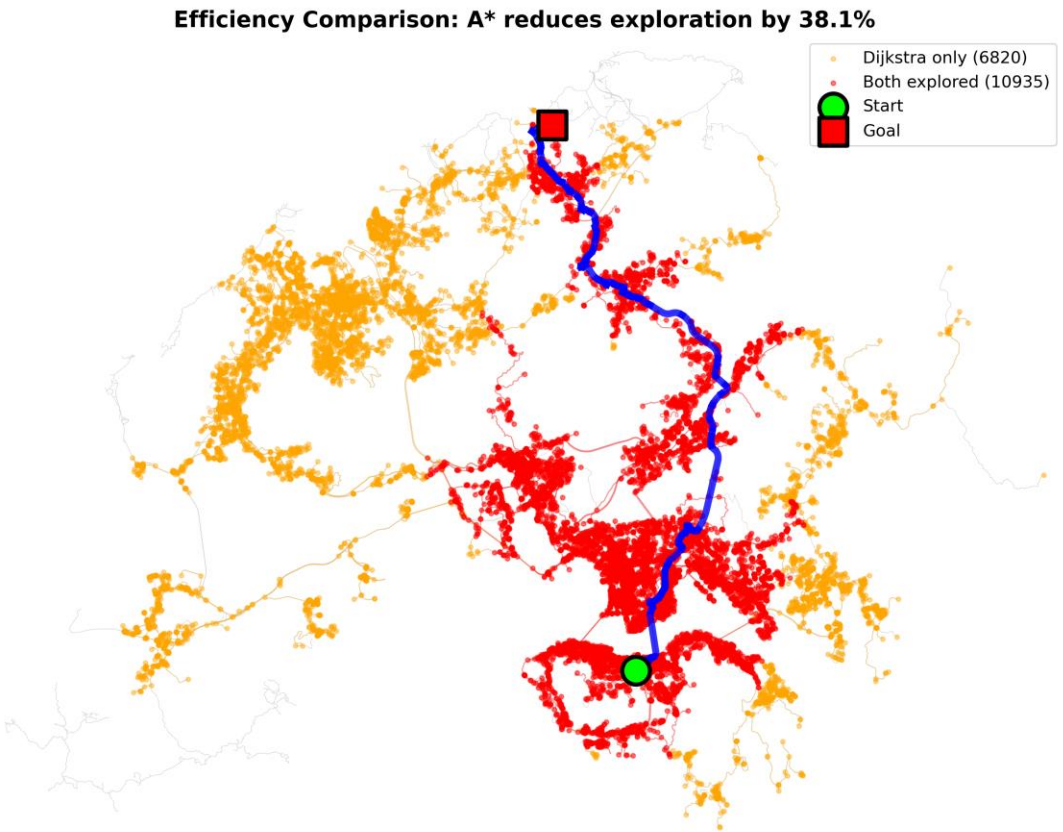
Feature	**Dijkstra**	**A***
Search Type	Uninformed	Heuristic
Priority Function	$g(n)$	$f(n) = g(n) + h(n)$
Nodes Explored	More	Fewer
Running Time	Slower	Faster
Optimality	Yes	Yes (if admissible)
Needs Goal Info	No	Yes

Further Topics

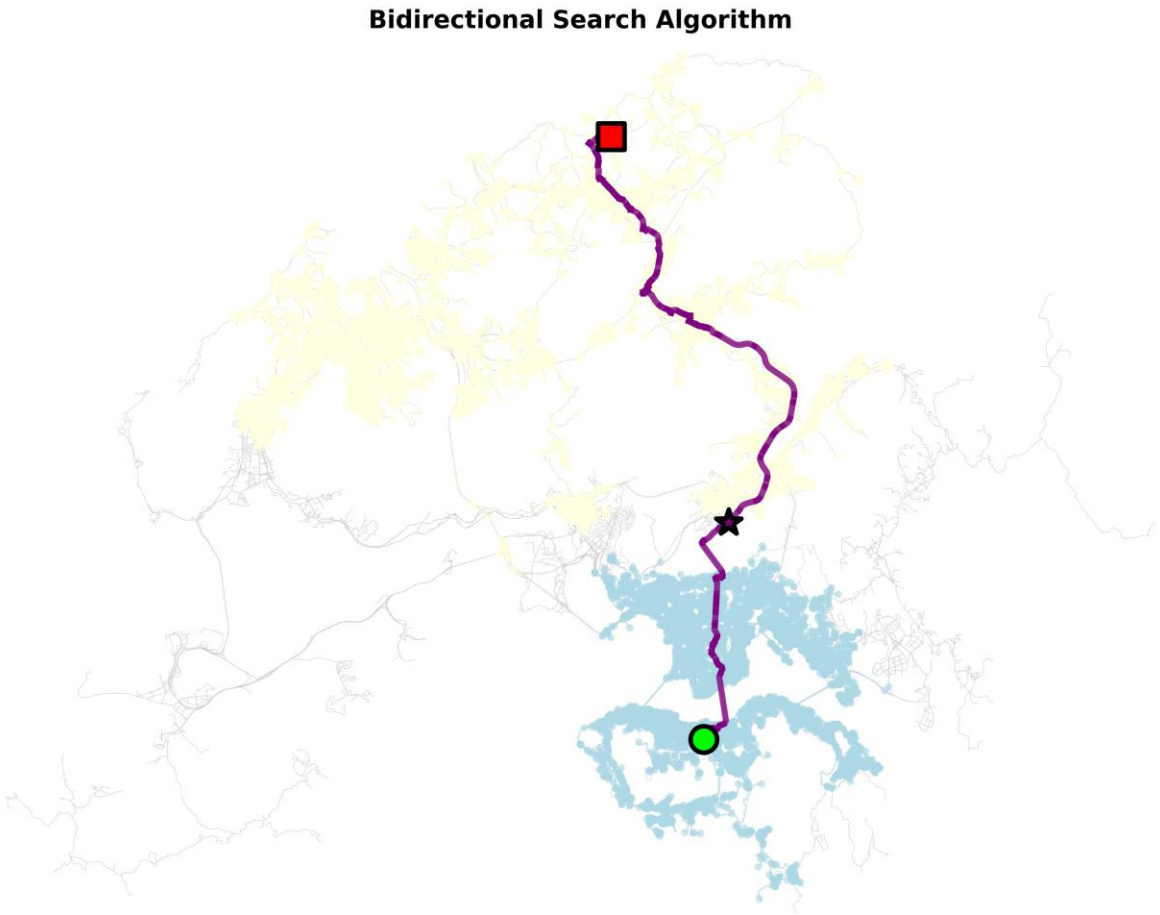
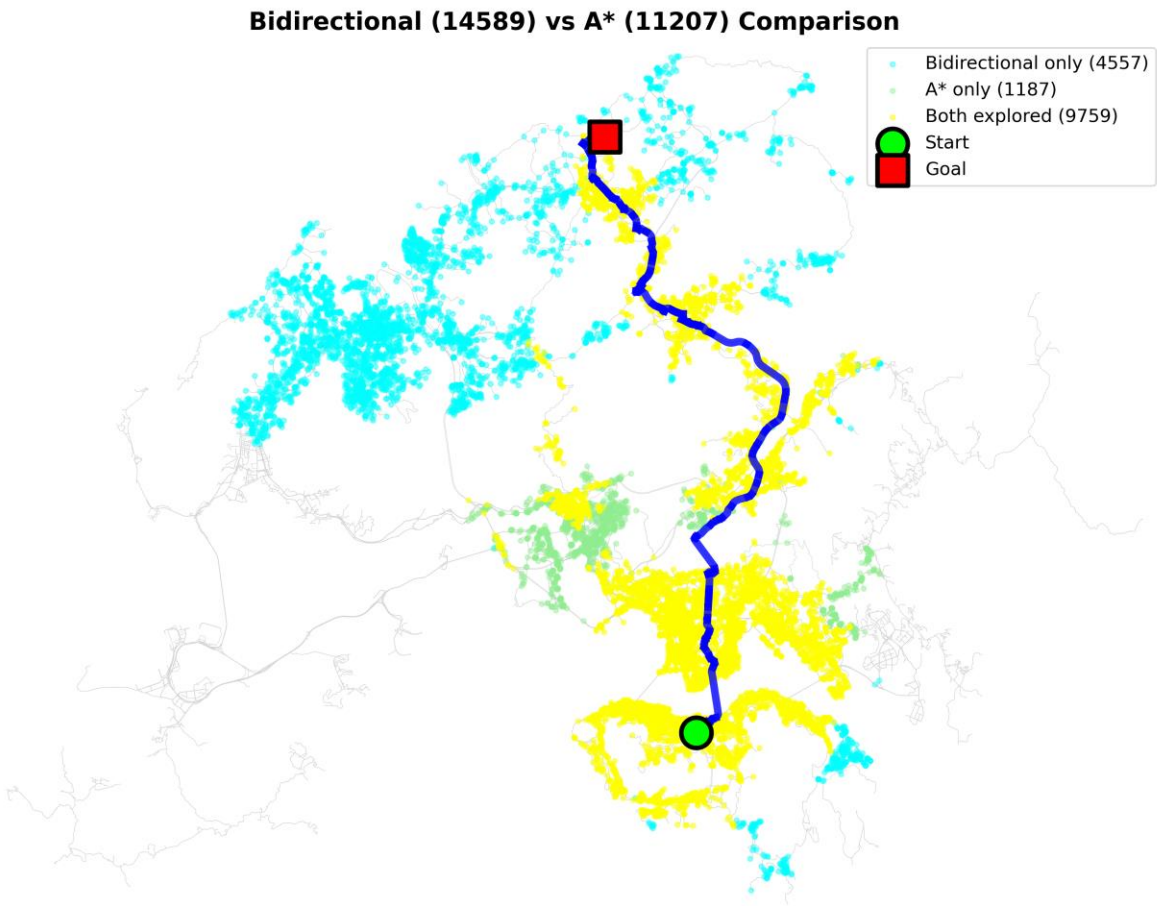
More Maps

You can try your codes on more maps to find the shortest path! Like how to go from Central to Lo Wu!





Further Topics

More Algorithms

Also there are **Bidirectional Search**, Weighted A*, D* for dynamic planning…